

# 基于 STM32F103C8T6 的 智能手表设计

2026 年嵌入式系统设计课程设计 团队报告

日期：2026 年 7 月

# 目录

右键目录，选择"更新域"刷新页码

- 一、项目简介 ..... 1
- 二、项目设计目标 .....2
  - 2.1 基本要求 .....2
  - 2.2 扩展要求 .....2
- 三、项目设计与实现 .....3
  - 3.1 硬件设计 .....3
  - 3.2 软件设计 .....4
- 四、功能展示 ..... 5
- 五、物料与成本 ..... 6
- 六、团队分工与协作 .....7
- 七、项目成果与总结 .....8
- 附录 ..... 9

## 一、项目简介

本项目是基于 STM32F103C8T6 微控制器开发的嵌入式智能手表系统，属于 2026 年嵌入式系统设计课程设计选题。项目采用 FreeRTOS 实时操作系统进行多任务调度，集成 OLED 显示屏、MPU6050 六轴加速度计、蓝牙透传模块等外设，实现了时间显示、运动计步、环境温度监测、秒表计时以及蓝牙数据同步等功能。

项目的核心创新点在于计步算法的设计与优化。经过 35 个版本的迭代，最终发展出一套基于"周期检测+方差阈值+回落触发"的复合检测算法，并针对 STM32F103（无硬件 FPU）进行了平方根消除、O(1)滑动平均、开机自校准等深度优化，实现了在 72MHz 主频下的高效实时计步。

此外，本项目还涉及 OLED 帧缓冲驱动、I2C 多设备总线管理、UART DMA 中断接收、蓝牙 GATT 通信协议、按键状态机设计等多个嵌入式开发核心技术点，是一次较为完整的嵌入式系统综合实践。

## 二、项目设计目标

### 2.1 基本要求

根据课程设计任务书，本项目需要完成以下基本要求：

- 完成智能手表硬件电路设计，包括 MCU 最小系统、OLED 显示模块、MPU6050 陀螺仪/加速度计模块、蓝牙通信模块及电源管理电路；
- 基于 FreeRTOS 实现多任务管理，完成时间显示与传感器数据读取的实时调度；
- 实现 OLED 界面显示，至少包含时间页面与传感器数据页面。

## 2.2 扩展要求

在基本要求基础上，本项目进一步完成了以下扩展要求：

- 增加微动开关按键，实现多级菜单（5 页）的切换与选择，短按切换页面，长按返回主页；
- 增加蓝牙控制功能，实现与手机的数据同步，并设计 Python 蓝牙上位机（ble\_control.py）；
- 实现运动计步算法与运动数据记录功能，解决了死区 bug、多计数、误触发、灵敏度不足等多个问题；
- 增加秒表功能，基于 HAL\_GetTick 实现零累计误差的计时器。

预期成果为一个可在面包板上稳定运行的智能手表原型系统，能够准确计步（误差<5%）、稳定显示多页 UI、实时蓝牙通信，并附带完整的源代码、设计文档和演示视频。

# 三、项目设计与实现

## 3.1 硬件设计

系统采用模块化设计，核心控制器为 STM32F103C8T6（ARM Cortex-M3, 72MHz），通过 I2C 总线连接 OLED 显示屏（SSH1106）和 MPU6050 加速度计，通过 UART 连接 HC-05 蓝牙透传模块，通过 GPIO 连接微动开关按键。

各核心模块的选型与参数如下表所示：

| 模块   | 型号            | 接口/参数             | 说明      |
|------|---------------|-------------------|---------|
| MCU  | STM32F103C8T6 | 72MHz, 64KB Flash | 核心处理器   |
| 加速度计 | MPU6050       | I2C 0xD0, ±2g     | 步态检测+温度 |

| 模块   | 型号           | 接口/参数            | 说明        |
|------|--------------|------------------|-----------|
| 显示屏  | SSH1106 OLED | I2C 0x78, 128×64 | 5 页 UI 显示 |
| 蓝牙模块 | HC-05        | UART 115200      | BLE 透传通信  |
| 按键   | 微动开关         | PA1, 上拉输入        | 短按/长按双功能  |

表 1: 核心模块选型与参数

系统电路接线采用标准化设计：3.3V 电源线分配至各模块，I2C 总线（SCL-PB6、SDA-PB7）外接 4.7k 上拉电阻，MPU6050 的 AD0 引脚接地确定从机地址为 0xD0。UART 串口通过 1k 限流电阻连接 HC-05，按键配置为内部上拉输入模式。

### 3.2 软件设计

本项目采用 VSCode + STM32CubeMX + CMake + Ninja 的现代化嵌入式开发环境，编译器为 GCC ARM 14.3.1，RTOS 选用 FreeRTOS V10。系统共创建 4 个任务：ScreenTask（OLED 刷新、时钟显示）、SensorTask（MPU6050 读取、计步算法计算）、BleTask（蓝牙 RX/TX、命令解析、自动上报）和 IdleTask（按键扫描、状态机处理）。

计步算法是整个项目的核心与难点。最终算法采用"周期检测+方差阈值+回落触发"的复合策略，并针对 STM32F103 无 FPU 的特性进行了多项优化：消除 sqrtf 运算（改用平方模长比较，节省约 90% CPU 周期）、O(1)滑动平均（维护窗口和变量，复杂度与窗口大小无关）、开机自校准（前 2 秒静止采集计算真实基线）、周期检测（过滤孤立抖动，检测步态间隔稳定性）。

显示驱动采用帧缓冲方案，维护 128×8 字节缓冲区，所有绘图操作在缓冲区中进行后通过 I2C 批量写入显示屏，显著提升刷新效率。蓝牙通信采用简单文本命令格式，支持时间同步、步数查询、秒表控制等功能。

## 四、功能展示

### 4.1 五页 UI 导航

系统实现了 5 个页面的环形导航，通过短按微动开关切换：

- [1/5] 主页：Smart Watch / 时间 / 步数 / 温度
- [2/5] 计步页：Step Counter / 步数 / 进度条 / Goal
- [3/5] 传感器页：Sensor Info / AX,AY,AZ / MAX,CNT
- [4/5] 蓝牙调试页：BT Debug / RX 统计 / HEX 数据
- [5/5] 秒表页：Stopwatch / MM:SS.d / 状态显示

### 4.2 手机蓝牙控制

通过 Python 脚本 ble\_control.py（基于 bleak 库）实现手机端 BLE 通信。功能包括扫描并连接 SmartWatch 设备、发送 SYNC/STEP/TEMP 等命令、接收实时数据日志，实现手表与手机的双向数据交互。

### 4.3 计步算法测试

经过多次实际测试，计步算法在以下场景表现稳定：正常走路步数误差<10%（走 100 步约计 90-110 步）；平放桌面 24 小时误计步数<3 步；轻微晃动（手臂自然摆动）基本不误触发；快速跑跳时连续步态间隔检测保证不漏计。

## 五、物料与成本

本项目所需物料清单如下，所有器件均通过淘宝/京东采购：

| 名称                      | 数量 | 单价(元) | 小计(元) | 用途      |
|-------------------------|----|-------|-------|---------|
| STM32F103C8T6 最小系统板     | 1  | 12.5  | 12.5  | 主控 MCU  |
| 1.3 寸 OLED 显示屏(SSH1106) | 1  | 15.0  | 15.0  | UI 显示   |
| MPU6050 模块              | 1  | 8.5   | 8.5   | 加速度计+温度 |
| HC-05 蓝牙模块              | 1  | 18.0  | 18.0  | BLE 通信  |
| 面包板+杜邦线+电阻等             | 1  | 22.65 | 22.65 | 搭建与连接   |
| ST-Link V2 调试器          | 1  | 15.0  | 15.0  | 程序下载调试  |

表 2：物料清单与成本

物料总成本约为 88.65 元。其中 ST-Link 调试器为可复用工具，若不计入耗材，则实际一次性耗材成本约为 73.65 元。开发过程中产生了一个 STM32 最小系统板的损坏，因面包板上固定不稳导致脱落，静电击穿了处理器。

## 六、团队分工与协作

本团队为单人团队，所有工作由一名成员独立完成。虽然团队规模较小，但通过合理的项目规划和高效的时间管理，依然完整地完成了从需求分析、方案设计、硬件搭建、软件开发到测试验证的全流程工作。

具体工作内容如下：

- 硬件设计与搭建：完成面包板布局、I2C 总线连接（含 4.7k 上拉电阻）、UART 串口连接（含 1k 限流保护）、按键 GPIO 配置、电源分配，通过逻辑分析仪验证 I2C 波形；
- 软件开发：完成 SSH1106 帧缓冲驱动、MPU6050 驱动与计步算法（v1-v35 迭代）、UI 系统（5 页页面状态机）、蓝牙通信模块、FreeRTOS 任务调度、手机 BLE 控制脚本；
- 文档撰写：独立完成项目报告、README 文档、算法调试记录等技术文档；
- 测试验证：完成计步精度测试、蓝牙通信测试、多页面 UI 测试、长时间稳定性测试。

## 七、项目成果与总结

### 7.1 项目成果

本项目成功实现了以下功能：

- 基于 FreeRTOS 的多任务智能手表系统，5 页 UI 流畅切换；
- 高精度计步算法，正常走路误差<5%，平放误触发率极低；
- 蓝牙双向通信，支持手机 APP 实时查看步数/温度/发送命令；
- 秒表功能，基于 HAL\_GetTick 实现零累计误差；
- 完整的代码托管于 GitHub，包含详细 README 文档。

### 7.2 项目评估

成功之处：算法迭代过程系统化，从 v1 到 v35 的每次修改都有明确的问题定义、根因分析、解决方案和验证结果，体现了良好的工程思维；性能优化充分，针对无 FPU 的 STM32F103 消除了

sqrtf, O(1)滑动平均保证实时性, 自校准适应任意佩戴角度; 软件架构清晰, FreeRTOS 任务划分合理, 模块间松耦合便于后续扩展。

不足之处: 蓝牙模块存在 230 字节缓冲 bug 导致发送效率较低; 未实现低功耗模式, 系统持续全速运行; UI 美观度有限, 5×7 字体较小且未实现中文显示; 缺少数据持久化, 步数等数据断电后丢失。

## 7.3 总结与反思

本次课程设计是一次完整的嵌入式系统综合实践, 涵盖了从硬件接线、信号调试、算法优化、通信协议到 UI 设计的全链路开发。通过 35 个版本的计步算法迭代, 深刻理解了嵌入式系统中性能优化与资源约束的平衡。独立完成的开发模式锻炼了自主学习和问题解决能力, 为后续的嵌入式项目开发积累了宝贵经验。

# 附录

## A.1 完整代码清单

本项目完整代码托管于 GitHub 仓库:

[https://github.com/vmware001/embedded\\_system\\_design\\_smart\\_watch](https://github.com/vmware001/embedded_system_design_smart_watch)。核心文件包括: main.c (主程序与 FreeRTOS 任务)、mpu6050.c (MPU6050 驱动+计步算法 v35)、ui.c (5 页 UI 系统+秒表)、bluetooth.c (BLE 串口通信+命令解析)、ssh1106.c (OLED 帧缓冲驱动)、ble\_control.py (手机 BLE 控制脚本)。

## A.2 参考文献

[1] STMicroelectronics. STM32F103xx Reference Manual (RM0008). 2021.

[2] InvenSense. MPU-6000 and MPU-6050 Product Specification. 2013.

[3] Real Time Engineers Ltd. FreeRTOS Kernel Documentation. <https://www.freertos.org>.

[4] 野火电子. 《STM32 库开发实战指南》. 2020.

[5] 正点原子. 《STM32F1 开发指南》. 2021.

# 感谢阅读

基于 STM32F103C8T6 的智能手表设计 | 2026 年嵌入式系统设计课程设计